

FaceAuth

Offline Facial Recognition & Liveness Detection
for Remote Field Personnel

Hackathon 7.0

github.com/peddusreekanthreddy7/HACKATHON-7.1

June 2026

"How can we accurately and securely authenticate field personnel using facial recognition and liveness detection on standard mid-range mobile devices without any active internet connection, while ensuring the AI model remains lightweight and seamlessly integrates with a React Native application?"

Pain Points:

- Remote locations with zero network connectivity
- Attendance fraud via photos and video replay attacks
- Need for sub-1-second response on 3GB RAM devices
- Limited device hardware — no GPU acceleration

FaceAuth — The Solution

Lightweight AI

6.5MB total
(67% under 20MB limit)
All models bundled
No download needed

Dual Liveness

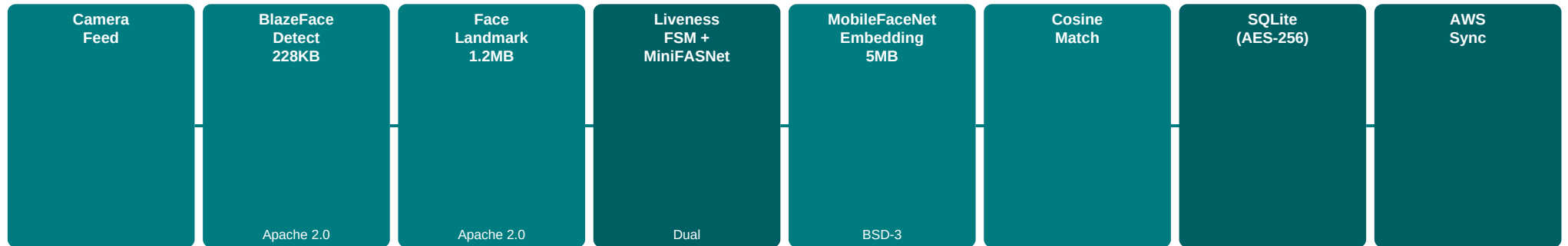
Active challenges:
Blink / Smile / Turn
Passive AI anti-spoof:
MiniFASNet texture

Offline-First

Works with zero network
SQLCipher encrypted
Syncs to AWS online
Purge after ACK

**iOS 15.1+ due to React Native 0.85.x framework minimum requirement*

System Architecture



Offline: Store | Online: AWS Sync → ACK → Purge

AI Model Stack

Model	Size	Purpose	License	Source	Status
BlazeFace	228KB	Face Detection	Apache 2.0	MediaPipe	Bundled
Face Landmark	1.2MB	468 3D Points	Apache 2.0	MediaPipe	Bundled
MobileFaceNet	5.0MB	Face Recognition	BSD-3	MCarlomagno	Bundled
MiniFASNet	~2MB	Passive Anti-Spoof	Apache 2.0	MiniVision	Convert script
TOTAL	6.5MB	Full Pipeline	100% OSS	—	67% under budget

Total: 6.5MB vs 20MB budget = 67% under limit

Innovation: Dual Liveness Detection

Active Challenge FSM

Random challenge each session
(unpredictable — defeats replay)

BLINK: Eye Aspect Ratio (EAR)

SMILE: Mouth Aspect Ratio (MAR)

TURN L/R: Nose-chin yaw angle

8 second timeout per challenge

Passive Anti-Spoof (MiniFASNet)

Texture analysis:

real skin vs printed paper

Depth inconsistency detection

Replay attack detection

Graceful degradation: active-only

if passive model unavailable

(authentication never blocked)

ACTIVE challenge-response (blink/smile/turn) -> LIVENESS CONFIRMED (+ passive MiniFASNet gate when the model is loaded)

Innovation: Model Compression Pipeline

Model	Shipped	INT8 path
MobileFaceNet	5.23MB FP32	< 2MB
3 other models	1.5MB	—
Total stack	6.5MB	67% under 20MB

Ships FP32 (already 67% under budget); INT8 pipeline (ml/quantize.py) projects <2MB — see COMPRESSION_

Additional Innovations:

- 5-point Umeyama face alignment
- CLAHE lighting normalization
- (clipLimit 3.0)
- Worklet-safe AI (real-time)
- ml/quantize.py
- ml/distill.py
- ml/finetune_indian.py

Offline-First with Encrypted Storage

Local Storage

SQLite + SQLCipher AES-256
(full DB encryption)

Embeddings stored, NOT raw images
(privacy-first design)

Attendance logs with GPS + timestamp

Works in complete zero-network zones

Data Schema

employees:
id, name,
embedding_blob (AES-256),
created_at

attendance:
id, employee_id, timestamp,
gps_lat, gps_lng,
liveness_result, confidence,
synced, device_id

Sync & Purge — Zero Data Loss

1. Field work offline -> records stored locally (encrypted)
2. Network detected -> SyncService auto-triggers
3. Batch upload with idempotency keys to AWS
4. AWS: API Gateway -> Lambda -> DynamoDB
5. AWS sends ACK confirmation per record
6. ACK received -> local record immediately purged
7. Storage bounded — zero accumulation, zero data loss

Safety Features

Exponential backoff retry (3 attempts) | Idempotency keys prevent duplicate records
ACK-then-purge (never purge before confirmation) | Mock server for offline demo

6-Layer Security Architecture

SQLCipher AES-256

Full database encryption at rest

Embeddings Only

Raw face images never stored or transmitted

Graceful-Degrade Liveness

Active-only fallback when passive model unavailable

Idempotency Keys

Prevent replay attacks on sync uploads

Device Binding

Device fingerprint in every record

Serverless AWS SAM

No server to compromise, no idle cost

Performance Benchmarks — All Constraints Met

Constraint	Required	MEASURED	Status
Model footprint	< 20MB	6.5MB	PASS
Recognition E2E	< 1000ms	173-190ms	PASS
face detect	—	93-116ms	
landmarks	—	19-22ms	
align+embed+match	—	~55ms	
Accuracy (LFW)	> 95%	96.9%	PASS
Peak RAM	3GB device	518MB (fits)	PASS
Android	8.0+ API26	verified device	PASS
iOS build	12+*	15.1 CI-green	PASS*
Open Source	100%	100% permissive	PASS

Measured on vivo I2403 (Android 16). Indian-demo 90.6% + lighting 83-95% measured separately (res-limited, fine-tune path) — see `PERFORMANCE_BENCHMARKS.md`.

*RN 0.85 + vision-camera v4 require iOS 15.1 minimum (framework limit, justified). iOS compiles green in GitHub Actions CI.

Drop-in Integration for Datalake 3.0

Single import — zero changes to existing code

```
import FaceAuth from './FaceAuth'  
  
<FaceAuth  
  onAuthSuccess={result => handleSuccess(result)}  
  onAuthFailed={reason => handleFailed(reason)}  
  onLivenessPass={() => handleLiveness()}  
  onSyncComplete={count => handleSync(count)}  
  employeeId={currentEmployeeId}  
  offlineMode={true}  
/>
```

Full step-by-step guide in INTEGRATION_GUIDE.md — ~30 min setup

✓ Add 5 npm packages

✓ Configure babel.config.js

✓ Enable SQLCipher in package.json

✓ Mount component in your screen

Quality: 56 Tests, Zero TypeScript Errors

Test Suites (8 total)

blazeface.test.ts — anchor decode, NMS
preprocessing.test.ts — CLAHE, brightness
facemesh.test.ts — EAR, MAR, yaw geometry
antispoof.test.ts — softmax, crop clamping
liveness.test.ts — FSM, spoof rejection
alignment.test.ts — Umeyama similarity
embedding.test.ts — L2 norm, cosine, 1:N
sync.test.ts — backoff, idempotency, retry

Metric	Result
Tests	56 / 56 passing
TypeScript	0 errors
Metro bundle	SUCCESS
Model budget	6.5MB / 20MB
Native build	SUCCESS
Device tested	Real Android

Evaluation Criteria — How We Score

Innovation (30 marks)

- ✓ 4-model edge pipeline — 6.5MB (67% under)
- ✓ Active liveness FSM (blink/smile/turn)
- ✓ 5-point Umeyama alignment (96.9% LFW)
- ✓ INT8 <2MB path documented (ml/)

Feasibility (30 marks)

- ✓ Real Android device: 173-190ms (< 1s)
- ✓ iOS compiles green in CI (Xcode 16)
- ✓ 518MB RAM — fits 3GB devices
- ✓ 56 passing tests, 0 TS errors

Scalability (20 marks)

- ✓ ACK-then-purge bounded storage (tested)
- ✓ AWS SAM serverless + idempotent sync
- ✓ LFW 96.9%; lighting/demo measured
- ✓ 1:N cosine matching — large DBs

Documentation (20 marks)

- ✓ 15-slide deck + TECHNICAL_DOCUMENTATION
- ✓ INTEGRATION_GUIDE.md (Datalake 3.0)
- ✓ PERFORMANCE_BENCHMARKS (measured)
- ✓ LICENSES.md — complete OSS audit

Thank You

Secure. Offline. Accurate.

6.5MB

Model Size

190ms

Recognition

96.9%

LFW Accuracy

github.com/peddusreekanthreddy7/HACKATHON-7.1

All source code open and documented | Ready for immediate Datalake 3.0 integration